

Last class:

- More Bash commands:
 - mv, cp, wc, sort, head, tail, find
- Wildcards
- Redirection
- Scripting
- Started with grep

Notes:

- Don't forget HW0
- HW1 posted, due October 2

Useful grep options:

- -i for ignore case (match capital and small letters). compare `grep meg LittleWomen.txt` and `grep -i meg LittleWomen.txt`

```
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$ grep -i meg LittleWomen.txt | wc -l
690
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$ grep meg LittleWomen.txt | wc -l
2
```

- -v to invert the search (find lines that don't match)
- -r to search recursively

```
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$ grep -ir elephant .
./littleWomen.txt:an elephant in silk and lace.
./littleWomen.txt:With elephantine grace,
./littleWomen.txt:young gentlemen would have liked better, but elephants could not have
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$ grep -iwr yesterday .
./littleWomen.txt:Yes, I did. I told you I'd make you pay for being so cross yesterday,
./littleWomen.txt:I made a 'pome' yesterday, when I was helping Hannah wash, and as
./littleWomen.txt:I telegraphed to your mother yesterday, and Brooke answered she'd come
./littleWomen.txt:office yesterday, for the doctor looked sober, and Hannah met took my
./littleWomen.txt:Yesterday at dinner, when Aunt was asleep and I was trying to be as still as a
./littleWomen.txt:yesterday's failure by a grand success today, so she ordered the
./littleWomen.txt:not only shut, but locked, and yesterday's mud still adorned the steps.
./littleWomen.txt:Yesterday at dinner, when an Austrian officer stared at us and then
./littleWomen.txt:"To New York. I had a bright idea yesterday, and this is it. You know
./littleWomen.txt:Yesterday was a quiet day spent in teaching, sewing, and writing in my
./littleWomen.txt:Yesterday that I was buttoning Amy's pinafore, and pulling your hair
./littleWomen.txt:daughter was true to the mother's teaching. I told Hanna so yesterday,
./littleWomen.txt:just as he is, and said yesterday that she thought poverty was a
./haiku.txt:Yesterday it worked
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$
```

Can use grep to reduce output from find:

```
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$ cd ..
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data$ find . | grep txt
./alkanes
./alkanes/pentane.pdb
./alkanes/get_headers.sh
./alkanes/lengths.txt
./alkanes/cubane.pdb
./alkanes/octane.pdb
./alkanes/ethane.pdb
./alkanes/propane.pdb
./alkanes/methane.pdb
./alkanes/headers.txt
./writing
./writing/LittleWomen.txt
./writing/haiku.txt
./writing/junk_text.txt
./writing/thesis
./writing/thesis/junk_text.txt
./numbers.txt
./project
./project/data
./project/results
./creatures
./creatures/basilisk.dat
./creatures/minotaur.dat
./creatures/unicorn.dat
./animal-counts
./animal-counts/animals.csv
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data$ find . | grep txt
./alkanes/lengths.txt
./alkanes/headers.txt
./writing/LittleWomen.txt
./writing/haiku.txt
./writing/junk_text.txt
./writing/thesis/junk_text.txt
./numbers.txt
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data$
```

Can then use this as input to other commands:

```
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data$ wc -l $(find . | grep txt)
 7 ./alkanes/lengths.txt
 4 ./alkanes/headers.txt
21022 ./writing/LittleWomen.txt
11 ./writing/haiku.txt
 1 ./writing/junk_text.txt
 1 ./writing/thesis/junk_text.txt
 5 ./numbers.txt
21051 total
```

`$(...)` construction tells Bash to evaluate this part first

Much of the power of grep comes from using regular expressions to allow complex search strings. See some examples of this on HW1

Some other useful shell commands/tools:

- ssh is command to open secure shell to another machine

```
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data$ ssh smaclachlan@riemann.math.mun.ca
smaclachlan@riemann.math.mun.ca's password:
Welcome to LabNet!
For support, contact labnet-support@mun.ca
For more info on LabNet services, visit https://www.labnet.mun.ca/

This is an SSH gateway server only. Do not compile code or run processes on this machine.

smaclachlan@riemann:~$ logout
Connection to riemann.math.mun.ca closed.
```

- logout or Ctrl-d to log out
- syntax is `ssh username@hostname.mun.ca`
- scp to copy remote files

```
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data$ scp smaclachlan@riemann.math.mun.ca:hw7_sample.py .
smaclachlan@riemann.math.mun.ca's password:
hw7_sample.py                                100% 1414   27.5KB/s   00:00
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data$ ls
alkanes  animal-counts  creatures  hw7_sample.py  numbers.txt  project  writing
```

- syntax is `scp username@hostname.mun.ca:filename .` or `scp filename username@hostname.mun.ca:pathname/.`

Can run a single command on a remote system using ssh:

```
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data$ ssh smaclachlan@riemann.math.mun.ca "ls w*"
smaclachlan@riemann.math.mun.ca's password:
values-(4B) @vfu
values-(4B).pvd
vanka-3db.py
vanka-latest.py
viz.tgz
vom_mesh_update.py
```

- use `top` or `htop` to show status of processes running on a machine
- use `ps` to show what's running in the current shell
- Can run jobs in "background" by putting `&` at end of the command

```
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$ grep -i meg LittleWomen.txt > meg.txt &
[1] 221998
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$
[1]+  Done                  grep --color=auto -i meg LittleWomen.txt > meg.txt
```

Useful when jobs will take a long time to run. Note that when a job runs in the background, we get a process number. Can use this to cancel job before it finished with `kill pid`. Can also get pid number from `top/htop`. Can also move the job from the background to the foreground with `fg pid` & back to background with `Ctrl-Z` to pause the current job, and then `bg` to move paused job to background.

Combine this with `nohup` command (short for "no hangup"). Normally all jobs associated with a login session end (are killed) when you log out. `nohup` command tells bash to let the job keep running after you log out.

Customize your bash environment by editing `.bash_profile` and `.bashrc`:

- No real need for 2 files, exists as a historical artifact. Usually redirect one to read the other.

```
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$ cat ~/.bash_profile
if [ -f ~/.bashrc ]; then
    source ~/.bashrc
fi
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$ cat ~/.bashrc
# ~/.bashrc: executed by bash(1) for non-login shells.
# See /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

# If not running interactively, don't do anything
case $- in
    *) ;;
    *) return;;
esac

# don't put duplicate lines or lines starting with space in the history.
# See bash(1) for more options
HISTCONTROL=ignoreboth

# append to the history file, don't overwrite it
shopt -s histappend

# for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
HISTSIZE=1000
HISTFILESIZE=2000

# check the window size after each command and, if necessary,
# update the values of LINES and COLUMNS.
shopt -s checkwinsize

# If set, the pattern "==" used in a pathname expansion context will
# match all files and zero or more directories and subdirectories.
#shopt -s globstar

# make less more friendly for non-text input files, see lesspipe(1)
[ -x /usr/bin/lesspipe ] && eval "$(SHELL=/bin/sh lesspipe)"

# Set variable identifying the chroot you work in (used in the prompt below)
if [ -z "${debian_chroot:-}" ] && [ -r /etc/debian_chroot ]; then
    debian_chroot=$(cat /etc/debian_chroot)
fi

# set a fancy prompt (non-color, unless we know we "want" color)
case "$TERM" in
    xterm-color*) _set_color_prompt=yes;;
    *) _set_color_prompt=no;;
esac

# uncomment for a colored prompt, if the terminal has the capability; turned
# off by default to not distract the user: the focus in a terminal window
# should be on the output of commands, not on the prompt
#force_color_prompt=yes

if [ -n "$force_color_prompt" ]; then
    if [ -x /usr/bin/dircolors ]; then
        test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" || eval "$(dircolors -b)"
        alias ls='ls --color=auto'
        #alias dir='dir --color=auto'
        #alias vdir='vdir --color=auto'

        alias grep='grep --color=auto'
        alias fgrep='fgrep --color=auto'
        alias egrep='egrep --color=auto'
    fi

    # colored GCC warnings and errors
    #export GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locus=01:quote=01'

    # some more ls aliases
    alias ll='ls -alF'
    alias la='ls -A'
    alias l='ls -CF'

    # Add an "alert" alias for long running commands. Use like so:
    # sleep 10; alert
    alias alert='notify-send --urgency=low -i "${if [ $? = 0 ]; then echo terminal || echo error}" "${history}|tail -n1|sed -e '\''s/^\s*[-@-9]\+\s*//;s/[:&|;]\+s/alerts/'\`}'"

    # Alias definitions.
    # You may want to put all your additions into a separate file like
    # ~/.bash_aliases, instead of adding them here directly.
    # See /usr/share/doc/bash-doc/examples in the bash-doc package.

    if [ -f ~/.bash_aliases ]; then
        . ~/.bash_aliases
    fi

    # enable programmable completion features (you don't need to enable
    # this, if it's already enabled in /etc/bash.bashrc and /etc/profile
    # sources /etc/bash.bashrc).
    if [ -f ~/.bash_completion ] && [ ! -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi

export TEXINPUTS=".:/"
export BIBINPUTS=".:/"

export EF_ALLOW_MALLOC_0=1

export OMP_NUM_THREADS=1
export OMPI_MCA_hwloc_base_binding_policy=none

eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_rsa.github
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$ cat ~/.bashrc | grep alias
# enable color support of ls and also add handy aliases
alias ls='ls --color=auto'
#alias dir='dir --color=auto'
#alias vdir='vdir --color=auto'

alias grep='grep --color=auto'
alias fgrep='fgrep --color=auto'
alias egrep='egrep --color=auto'

# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'

# Add an "alert" alias for long running commands. Use like so:
# sleep 10; alert
alias alert='notify-send --urgency=low -i "${if [ $? = 0 ]; then echo terminal || echo error}" "${history}|tail -n1|sed -e '\''s/^\s*[-@-9]\+\s*//;s/[:&|;]\+s/alerts/'\`}'"

# ~/.bash_aliases, instead of adding them here directly.
if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

Use alias to write shortcuts. alias 'ls='ls --color=auto' turns on colors by default for ls.
```

Use `export` to set environment variables and paths. Can also use these at the command line. To view environment variables, use `set`

```
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$ set | grep PATH
BASH_LOADABLES_PATH=/usr/local/lib/bash:/usr/lib/bash:/opt/local/lib/bash:/usr/pkg/lib/bash:/opt/pkg/lib/bash:
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
if [ [ -z ${CPATH:-} ] || $cur == ?(.(?)/.* ) ]; then
    for i in ${CPATH}:/;
do
    local PATH=$PATH:/usr/sbin:/usr/local/sbin type $1 && /dev/null
    COMPREPLY+=($comgen -W "$($PATH=$PATH:/usr/sbin) lsmcd | awk '{if (NR == 1) print $1}'" -- "$1")
    local PATH=$PATH:/usr/sbin;
    local PATH=$PATH:/usr/sbin;
    COMPREPLY+=($comgen -W "$($PATH=$PATH:/usr/sbin) lspci -n | awk '{print $3}'" -- "$cur")
    local PATH=$PATH:/usr/sbin;
    COMPREPLY+=($comgen -W "$($PATH=$PATH:/usr/sbin) lsusb | awk '{print $6}'" -- "$cur")
fi
scottmac@kaczmarz:~/CMS60950/shell-lesson-data/exercise-data/writing$ set | grep PATH
BASH_LOADABLES_PATH=/usr/local/lib/bash:/usr/lib/bash:/opt/local/lib/bash:/usr/pkg/lib/bash:/opt/pkg/lib/bash:
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
if [ [ -z ${CPATH:-} ] || $cur == ?(.(?)/.* ) ]; then
    for i in ${CPATH}:/;
do
    local PATH=$PATH:/usr/sbin:/usr/local/sbin type $1 && /dev/null
    COMPREPLY+=($comgen -W "$($PATH=$PATH:/usr/sbin) lsmcd | awk '{if (NR == 1) print $1}'" -- "$1")
    local PATH=$PATH:/usr/sbin;
    local PATH=$PATH:/usr/sbin;
    COMPREPLY+=($comgen -W "$($PATH=$PATH:/usr/sbin) lspci -n | awk '{print $3}'" -- "$cur")
    local PATH=$PATH:/usr/sbin;
    COMPREPLY+=($comgen -W "$($PATH=$PATH:/usr/sbin) lsusb | awk '{print $6}'" -- "$cur")
fi

Could do more with command line, but we'll leave it here for now.
```

Python is computer language

- designed as a "general purpose" language
- emphasis on code readability
- widely adopted in scientific computing and other areas (AI/ML, web development)

We can run Python codes in several ways.

- Often run at command line, particularly when running on servers or HPC resources
- Can also run codes directly through an Integrated Development Environment (IDE)
 - colab, research.google.com "Google colab" cloud computing environment
 - Install JupyterLab on your computer (conda, homebrew...)
 - Use IDE with Jupyter support.

For this course, I'll consistently use Visual Studio Code as my IDE, with python and Jupyter support. Free download, Python and Jupyter extensions. Will add more extensions as we go.

Look at VSCode interface

- can add text (markdown) and code blocks
- outline, line numbers, etc.
- Add more text in markdown
- Add some math in code/docs

In [1]: `25+3`

Out[1]: `75`

- enter gives new line in same code block, shift-enter moves to next block, executing code if appropriate
- Note difference between opening as a Jupyter notebook and a python script. These are two different things!

Basic Markdown commands:

- use `*` or `=` to create bulleted lists
- use `1.` to create numbered lists
- use `#` or `##` to create headings
- use `$(...)` to render math using LaTeX: $\sum_{i=1}^N i = \frac{N(N+1)}{2}$

- use `code` to highlight a single command
- use `text` to highlight several lines of code or inputs/outputs

Programming in Python:

- start with variables
- variable names can contain letters, numbers, and underscores, but no spaces, dashes, or other punctuation
- variable names are case sensitive (`age` and `Age` are 2 different variables)
- variable names cannot start with a number. By convention, start with underscore only for certain reasons
- create variables and assign values with `=`

In [2]: `age = 25`
`test = Age = 28`
`name = "Scott"`

Use `print` to print variables

In [3]: `print(Age)`
`print(age)`
`print(name)`
`print(name, 'is', age, 'years old')`

28
25
Scott

Use `paneL` in VSCode to display variable information

Have to create variable names before you use them

In [4]: `print(last_name)`

```
NameError                                 Traceback (most recent call last)
Cell In[4], line 1
----> 1 print(last_name)

NameError: name 'last_name' is not defined

Can do basic math with numerical variables
```

In [5]: `age + Age`
`test + age`
`ratio = test/age`

Note that only get output from last line

In [6]: `age + Age`

Out[6]: `53`

In [7]: `test/age`

Out[7]: `1.12`

Variables have different types

- integers (int), floating-point numbers (float/double)
- strings (str)

Treat strings as lists of characters

- each position is indexed by a number, starting from 0

In [8]: `name[0]`

Out[8]: `'s'`

In [9]: `name[2]`

Out[9]: `'o'`

How do we know how many characters are in a string? Use `len` to find its length

In [10]: `len(name)`

Out[10]: `5`

Can "slice" a string to get more than one character

In [11]: `long_string = "I like fuzzy dogs!"`
`long_string[0:10]`

Out[11]: `'I like fuz'`

In [12]: `long_string[8:-4]`

Out[12]: `'uzzy d'`

Positive indices count from beginning (0 is first character), negative indices count from end (cannot use -0 for last character). Use empty index to go from first or to the last

In [13]: `long_string[6:]`

Out[13]: `'fuzzy dogs!'`

In [14]: `long_string[:5]`

Out[14]: `'I like fuzzy '`

Can optionally specify a step

In [15]: `long_string[2:14:2]`

Out[15]: `'lk uz '`

In [16]: `long_string[5:21:-1]`

Out[16]: `'eki'`

Use negative steps to go backwards, but then reverse start/stop order

IDE can show us a table of existing variables and their types, but we can also ask python for them directly

In [17]: `type(Age)`

Out[17]: `int`

In [18]: `type(long_string)`

Out[18]: `str`

Data types are important because they control what can or cannot do with the variables

In [19]: `Age+age`

Out[19]: `3`

In [20]: `long_string+name`

```
TypeError                                 Traceback (most recent call last)
Cell In[20], line 1
----> 1 long_string+name

TypeError: unsupported operand type(s) for +: 'str' and 'str'

Cannot subtract or divide strings! But can add or multiply them
```

In [21]: `string1 = "Ha"`
`string2 = "a"`
`print(string2 * 2+string1)`

Out[21]: `HaHaHa`

In [22]: `string3 = "Hem"`
`print(string1*16 + string2 * string3)`

Out[22]: `HaHaHaHaHaHaHaHaHaHaHaHaHaHaHaHa`

Adding strings concatenates them

Multiplying a string by an integer produces a string repeating the first that number of times

Can multiply a string by a number, but you can't otherwise combine them

In [23]: `print(1+2*5)`

```
TypeError                                 Traceback (most recent call last)
Cell In[23], line 1
----> 1 print(1+2*5)

TypeError: unsupported operand type(s) for +: 'int' and 'str'

Can, however, convert numbers to strings and vice-versa
```

In [24]: `print(str(1)+2*5)`

Out[24]: `12`

In [25]: `print(1+int('2'))`

Out[25]: `3`

Python variables take static values

In [26]: `var1 = 2`
`var2 = 5+var1`

Out[26]: `7`

In [27]: `print(var2)`

Out[27]: `7`

In [28]: `var1 = 1`
`print(var2)`

Out[28]: `7`

Dependency of `var2` on `var1` is resolved at assignment, changing `var1` later on does not change `var2`